Movie Recommendation & Review System

No-SQL-Based Project Documentation ,Project 8

Team 16:

Team Members	**Roll No**
Souvik Mondal	**UG/SOET/30/24/517**
Bijay Das	**UG/SOET/30/24/537**
Suman Ghosh	**UG/SOET/30/24/570**
Priya Kumari	**UG/SOET/30/24/601**
Sumit Kumar	**UG/SOET/30/24/620**

INDEX

List	Topic
1	Project Overview
2	Objective
3	System Architecture
4	Data Modeling & Strategy
5	CRUD Operations
6	Aggregation Pipeline
7	Indexing
8	Setup & Installation
9	Conclusion

1. PROJECT OVERVIEW

The Movie Recommendation & Review System is a **NoSQL-based application** designed to store, manage, and analyze movie data, user reviews, and ratings. The system allows:

- User authentication with JWT
- Browsing and searching movies
- Adding reviews and ratings
- Leaderboard insights for top-rated movies

The project uses **MongoDB** as a document-oriented NoSQL database to handle unstructured and semi-structured movie data efficiently.

Repository Structure:

- **backend/** → Flask API with routes for auth, movies, reviews
- **frontend/** → HTML/CSS/JavaScript SPA
- **config/** → Database bootstrap
- **routes/** → Modular route blueprints
- **package.json**

2. OBJECTIVE

- Store and analyze movie records using MongoDB collections: `users`, `movies`, `reviews`
- Provide recommendation and leaderboard features
- Support CRUD operations for movies and reviews
- Implement aggregation pipelines for analytics (e.g., top-rated movies)
- Use indexing for performance

3. SYSTEM ARCHITECTURE

Frontend: HTML / CSS / JavaScript (SPA with hash-based routing) Backend: Python Flask Database: MongoDB (NoSQL) Deployment: Localhost / Cloud (Vercel for frontend, Flask backend API)

Collections Overview:

- **users** → Stores user profiles and authentication data
- **movies** → Stores movie details (title, genre, release year, etc.)
- **reviews** → Stores user reviews linked to movies

4. DATA MODELING AND STRATEGY

Movies Collection (Embedding + Referencing)

- **Rationale:** Movie metadata (title, genre, release year, description) is stored directly in the movie document. Reviews are referenced separately to avoid bloating the movie record.
- **Fields:**
  - `title`
  - `genre`
  - `release_year`
  - `description`
  - `director`
  - `cast` (array of strings)

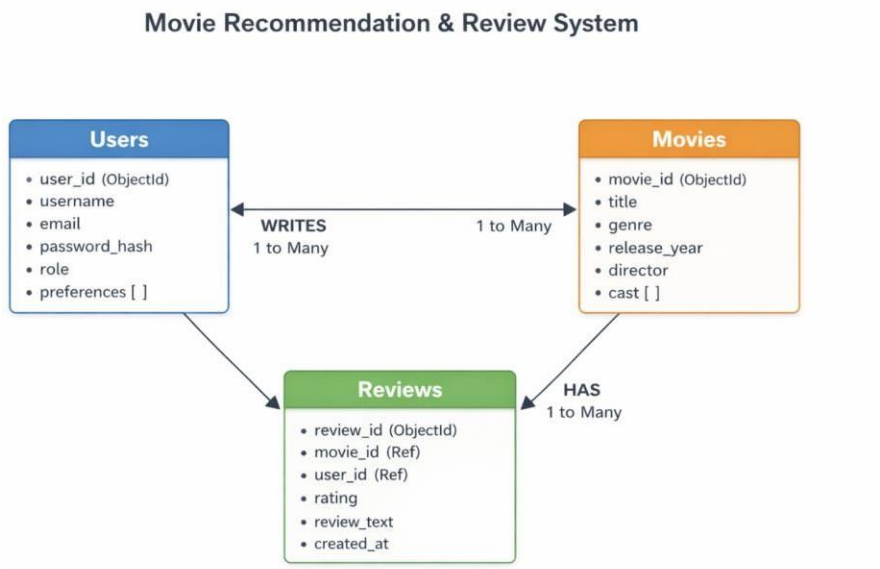
Reviews Collection (Referencing)

- **Rationale:** Reviews are stored in a separate collection and reference both `movie_id` and `user_id`. This allows independent lifecycles for reviews and avoids redundant updates when movies or users change.
- **Fields:**
  - `movie_id` (ObjectId reference to Movies)
  - `user_id` (ObjectId reference to Users)
  - `rating` (1–5)
  - `review_text`
  - `created_at`

Users Collection (Embedding)

- **Rationale:** User profile and authentication details are embedded directly in the user document for quick access. Session tokens can be stored here as well.
- **Fields:**
  - `username`
  - `email`
  - `password_hash`
  - `role` (user/admin)
  - `preferences` (array of genres)

ER Diagram (Conceptual)



- **Users ↔ Reviews ↔ Movies**
  - A user can write many reviews.
  - A movie can have many reviews.
  - Reviews link users and movies via references.

5. CRUD OPERATIONS

The system exposes a versioned RESTful API for all core resources — **Movies**, **Users**, and **Reviews** — using **JavaScript** and **MongoDB Atlas**.

5.1 Create (Insert Documents)

Insert a Movie:

```
const movie = {
  movie_id: "M001",
  title: "Inception",
  genre: "Sci-Fi",
  release_year: 2010,
  director: "Christopher Nolan",
  cast: ["Leonardo DiCaprio", "Joseph Gordon-Levitt", "Elliot Page"]
};

db.movies.insertOne(movie);
```

Insert a User:

```
const user = {
  user_id: "U001",
  username: "bijaydas",
  email: "bijay@example.com",
  preferences: ["Action", "Thriller"]
};

db.users.insertOne(user);
```

Add a Review:

```
const review = {
  review_id: "R001",
  movie_id: "M001",
  user_id: "U001",
  rating: 5,
  review_text: "Mind-bending masterpiece!",
  created_at: new Date()
};

db.reviews.insertOne(review);
```

5.2 Read (Query Documents)

Find All Movies:

```
const movies = db.movies.find();
movies.forEach(m => { print(m.title, m.genre); });
```

Find Reviews for a Movie:

```
db.reviews.find({ movie_id: "M001" }, { user_id: 1, rating: 1, review_text: 1 });
```

Find Top-Rated Movies:

```
db.reviews.aggregate([
  { $group: { _id: "$movie_id", avg_rating: { $avg: "$rating" } } },
  { $sort: { avg_rating: -1 } },
  { $limit: 5 }
]);
```

5.3 Update (Modify Documents)

Update Movie Details:

```
db.movies.updateOne(
  { movie_id: "M001" },
  { $set: { genre: "Science Fiction", director: "C. Nolan" } }
);
```

Update Review Rating:

```
db.reviews.updateOne(
  { review_id: "R001" },
  { $set: { rating: 4, review_text: "Still amazing, but not perfect." } }
);
```

5.4 Delete (Remove Documents)

Delete a Review:

```
db.reviews.deleteOne({ review_id: "R001" });
```

Remove a Movie Record:

```
db.movies.deleteOne({ movie_id: "M001" });
```

Delete a User Account:

```
db.users.deleteOne({ user_id: "U001" });
```

6. AGGREGATION PIPELINES

To meet the requirement for **movie-wise analytics**, the system utilizes the **MongoDB Aggregation Pipeline**. This processes data within the database engine to reduce network overhead and provides insights like top-rated movies and most active reviewers.

6.1 Top-Rated Movies

Uses `$group`, `$sort`, and `$lookup` stages to calculate average ratings per movie and return the highest-rated titles.

```
const pipelineTopRatedMovies = [
  { $group: { _id: "$movie_id", avg_rating: { $avg: "$rating" } } },
  { $sort: { avg_rating: -1 } },
  { $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movie_id",
      as: "movie_info"
  }},
  { $project: {
      movie_id: "$_id",
      title: { $arrayElemAt: ["$movie_info.title", 0] },
      genre: { $arrayElemAt: ["$movie_info.genre", 0] },
      avg_rating: 1,
      _id: 0
  }}];
const result = db.reviews.aggregate(pipelineTopRatedMovies);
```

6.2 Most Active Reviewer

Counts reviews per user → sorts → returns the most active reviewer.

```
const pipelineMostActiveReviewer = [
  { $group: { _id: "$user_id", review_count: { $sum: 1 } } },
  { $sort: { review_count: -1 } },
```

```
  { $limit: 1 },
  { $lookup: {
      from: "users",
      localField: "_id",
      foreignField: "user_id",
      as: "user_info"
  }},
  { $project: {
      user_id: "$_id",
      username: { $arrayElemAt: ["$user_info.username", 0] },
      email: { $arrayElemAt: ["$user_info.email", 0] },
      review_count: 1,
      _id: 0
  }}
];


const mostActive = db.reviews.aggregate(pipelineMostActiveReviewer);
```

Genre Popularity

Calculates the number of reviews per genre to identify trending categories.

```
const pipelineGenrePopularity = [
  { $lookup: {
```

```
    from: "movies",
    localField: "movie_id",
    foreignField: "movie_id",
    as: "movie_info"
  }},
  { $unwind: "$movie_info" },
  { $group: { _id: "$movie_info.genre", total_reviews: { $sum: 1 } } },
  { $sort: { total_reviews: -1 } }
];
const genreStats = db.reviews.aggregate(pipelineGenrePopularity);
```

7. INDEXING

Indexing improves performance for frequently queried fields like `movie_id`, `user_id`, and `rating`.

7.1 Create Indexes

Index on movie_id (reviews):

```
db.reviews.createIndex(
  { movie_id: 1 },
  { name: "idx_movie_id" }
);
```

Compound Index (movie_id + rating):

```
db.reviews.createIndex(
  { movie_id: 1, rating: -1 },
  { name: "idx_movie_rating" }
);
```

Index on user_id (reviews):

```
db.reviews.createIndex(
  { user_id: 1 },
  { name: "idx_user_id" }
);
```

Index on Embedded Field (preferences in users):

```
db.users.createIndex(
  { "preferences": 1 },
  { name: "idx_user_preferences" }
);
```

8. Setup & Installation

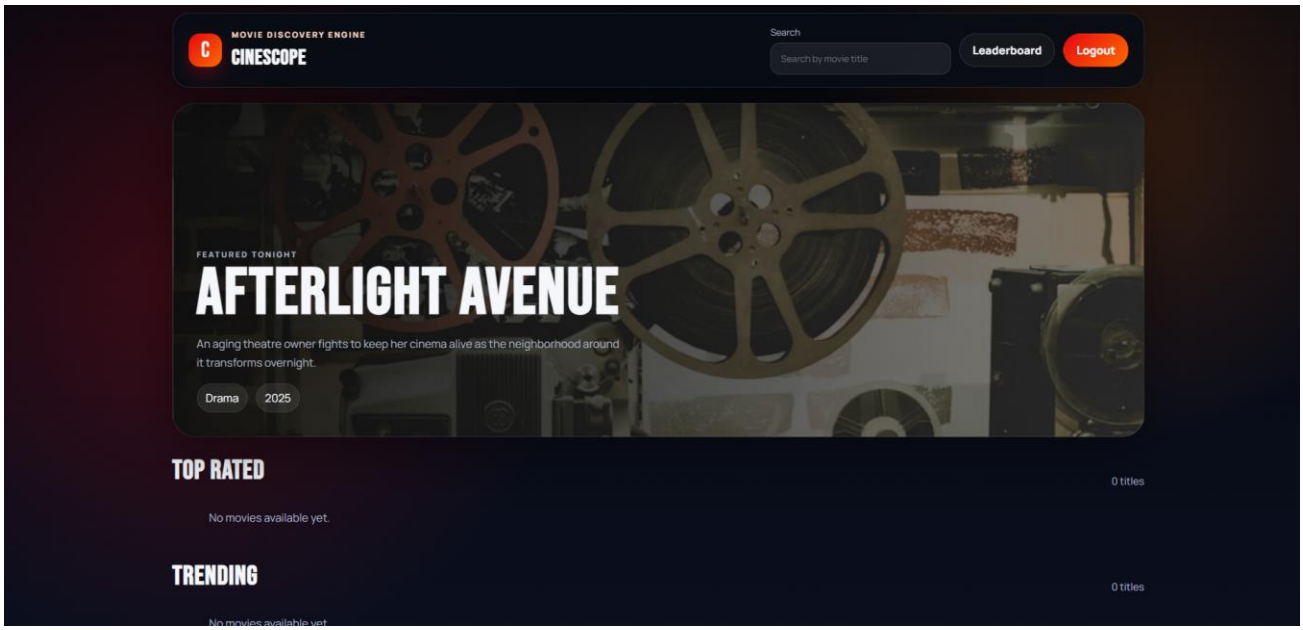
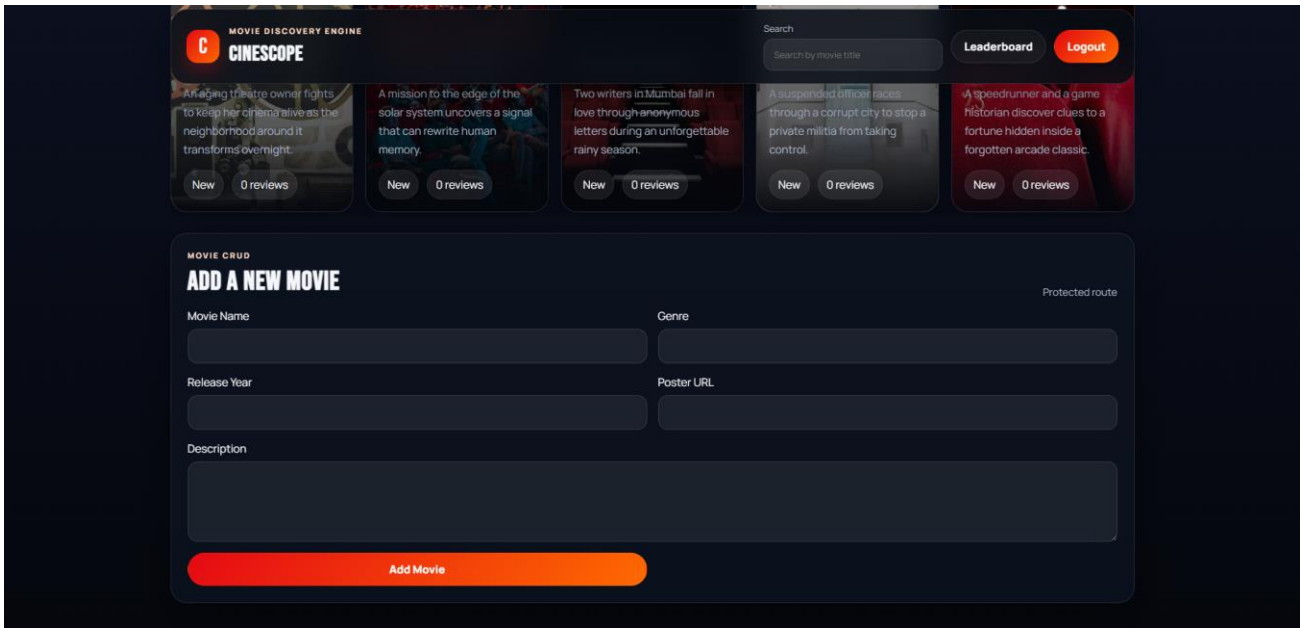
Prerequisites:

- Python 3.10+
- MongoDB 6.x (local or Atlas)
- Node.js (for npm scripts)

Steps:

1. Clone repo
2. Create virtual environment
3. Install dependencies (`pip install -r requirements.txt`)
4. Configure `.env` with `MONGO_URI`, `SECRET_KEY`, `PORT`
5. Run backend: `python app.py`
6. Run frontend: `npm run start`
7. Access app at http://localhost:5000

9. Conclusion

The Movie Recommendation & Review System demonstrates how MongoDB can be used to manage entertainment data efficiently. With modular collections, CRUD operations, aggregation pipelines, and indexing, the system provides a scalable solution for movie discovery, reviews, and analytics.

10. Screenshots & User Interface